University of East London

Task-3 Report- Artificial Intelligence & Machine Vision

Table of Contents

1.Introduction 3

2. Dataset Description 4

2.1 Dataset Overview 4

2.2 Data Preprocessing 4

2.3 Dataset Augmentation 6

3. Methodology 7

3.1 Data Preprocessing and Augmentation 7

3.2 Model Architecture 7

3.3 Transfer Learning 8

4. Model Training 9

4.1 Training Process 9

4.2 Validation During Training 10

4.3 Monitoring Performance 11

5. Results 12

5.1 Accuracy and Loss 12

5.3 Class Distribution 13

6. Discussion 14

6.1 Model Performance Analysis 14

6.2 Future Improvements 14

6.3 Application in Real-World Scenarios 14

7. Conclusion 15

8. References 16

1.Introduction

This paper examines the feasibility of using Convolutional Neural Networks (CNNs) in image classification use case and aims at categorizing plant species by images. CNNs are one of the popular categories of deep learning algorithms used in computer vision since it enables the model to learn features directly from images. In this project, we utilized the AlexNet model, which is a famous CNN with a deep structures and that perform very well. Among the images included in the dataset are images of assorted plants belonging to different species because the main objective of the experiment is to build a model that will be capable of identifying the species of plants selected at the later stages from the images of plants provided during the test stage.

The main goal of the project was to apply transfer learning in which a model trained on a massive amount was adjusted for the targeted task. Specifically, the last fully connected layer of the deep neural network of the AlexNet as adopted for this purpose at the beginning was pruned and modified to match the amount of data contained in the classification set of plants. That is why after keeping a part of the dataset for testing, the model was trained, validated, and tested in terms of its accuracy in classification of plant species.

2. Dataset Description

2.1 Dataset Overview

[redacted]

```
% Setup Paths
datasetPath = fullfile(pwd, 'Mydataset', 'split_ttv_dataset_type_of_plants');
imds = imageDatastore(datasetPath, 'IncludeSubfolders', true, 'LabelSource', 'foldernames');

% Create Figures Folder
if ~exist('Figures', 'dir')
    mkdir('Figures');
end
```

[redacted]

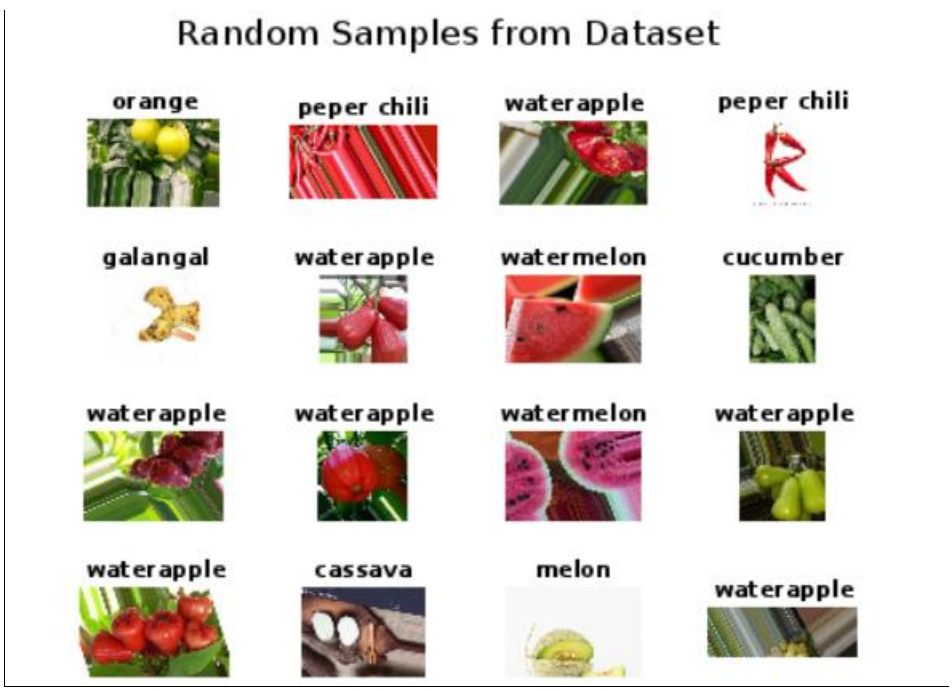
[redacted]

[redacted]

```
figure;
perm = randperm(numel(imds.Files), 16); % Random 16 images
for i = 1:16
    subplot(4,4,i)
    img = readimage(imds, perm(i));
    imshow(img)
    title(string(imds.Labels(perm(i))), 'FontSize', 8);
end
sgtitle('Random Samples from Dataset');
saveas(gcf, fullfile('Figures', 'Random_Samples.png'));
```

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
figure;
perm = randperm(numel(imds.Files), 16); % Random 16 images
for i = 1:16
    subplot(4,4,i)
    img = readimage(imds, perm(i));
    imshow(img)
    title(string(imds.Labels(perm(i))), 'FontSize', 8);
end
sgtitle('Random Samples from Dataset');
saveas(gcf, fullfile('Figures', 'Random_Samples.png'));
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

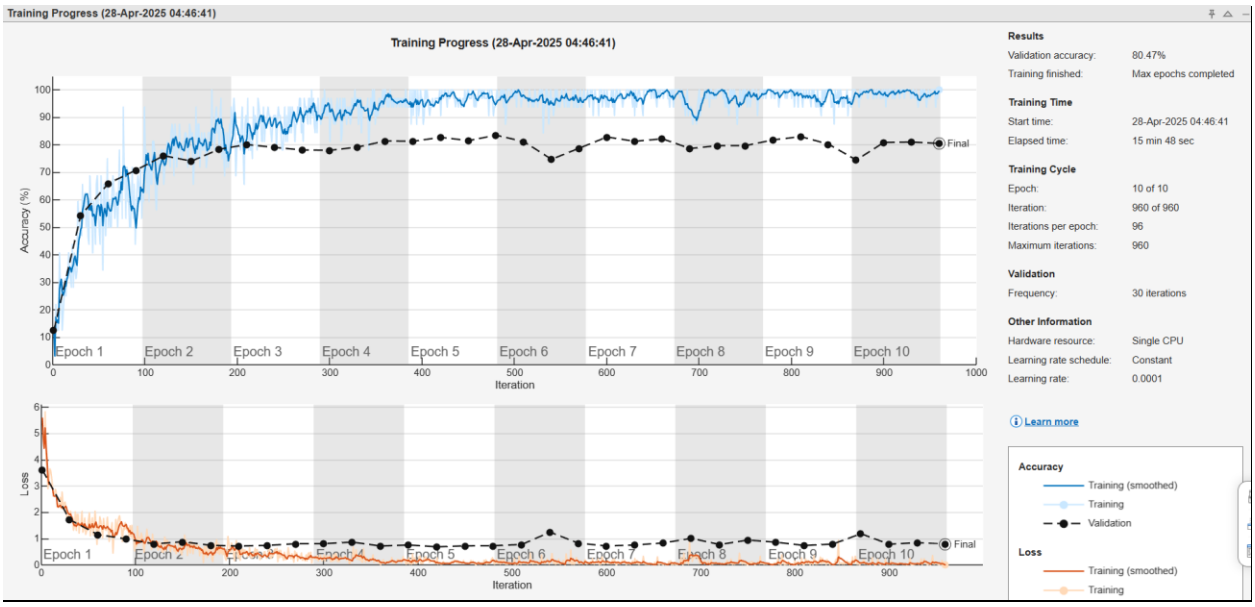
[redacted]

```
net = alexnet;
layers = net.Layers;
numClasses = numel(categories(imds.Labels));

% Replace last fully connected layer and classification laye
layers(end-2) = fullyConnectedLayer(numClasses);
layers(end) = classificationLayer;

%% Step 3: Set Training Options
options = trainingOptions('adam', ...
    'InitialLearnRate', 1e-4, ...
    'MaxEpochs', 10, ...
    'MiniBatchSize', 32, ...
    'ValidationData', augimdsTest, ...
    'ValidationFrequency', 30, ...
    'Plots', 'training-progress', ...
    'Verbose', true);

%% Step 4: Train the Network
trainedNet = trainNetwork(augimdsTrain, layers, options);
```

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

```
layers(end-2) = fullyConnectedLayer(numClasses);
layers(end) = classificationLayer;

%% Step 3: Set Training Options
options = trainingOptions('adam', ...
    'InitialLearnRate', 1e-4, ...
    'MaxEpochs', 10, ...
    'MiniBatchSize', 32, ...
    'ValidationData', augimdsTest, ...
    'ValidationFrequency', 30, ...
    'Plots', 'training-progress', ...
    'Verbose', true);
```

[redacted]

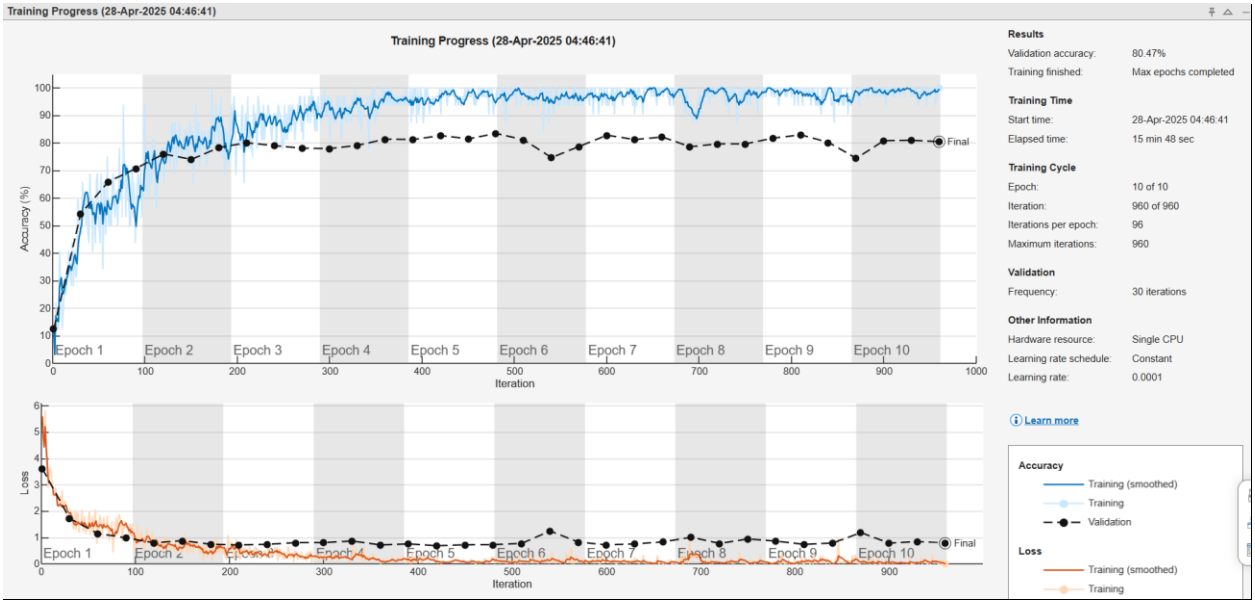
[redacted]

```
Command Window
|      10 |         930 |       00:15:12 |       96.88% |       81.00% |       0.0788 |       0.8409 |      1.0000e-04 |
|      10 |         950 |       00:15:29 |       96.88% |              |       0.0880 |              |      1.0000e-04 |
|      10 |         960 |       00:15:43 |      100.00% |       80.47% |       0.0018 |       0.7819 |      1.0000e-04 |
|=========================================================================================================|
Training finished: Max epochs completed.
Test Set Accuracy = 80.47%
>>
```

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

```
%% Step 5: Evaluate the Model
YPred = classify(trainedNet, augimdsTest);
YTest = imdsTest.Labels;

accuracy = sum(YPred == YTest) / numel(YTest);
fprintf('Test Set Accuracy = %.2f%%\n', accuracy*100);

fileID = fopen('accuracy.txt','w');
fprintf(fileID, 'Test Set Accuracy = %.2f%%\n', accuracy*100);
fclose(fileID);
```

[redacted]

[redacted]

[redacted]

[redacted]

```
% Save training progress figure
trainingFig = findall(groot,'Type','Figure','Name','Training Progress');
if ~isempty(trainingFig)
    saveas(trainingFig, 'trainingProgress.png');
end

%% Step 5: Evaluate the Model
YPred = classify(trainedNet, augimdsTest);
YTest = imdsTest.Labels;

accuracy = sum(YPred == YTest) / numel(YTest);
fprintf('Test Set Accuracy = %.2f%%\n', accuracy*100);

fileID = fopen('accuracy.txt','w');
fprintf(fileID, 'Test Set Accuracy = %.2f%%\n', accuracy*100);
fclose(fileID);
```

[redacted]

[redacted]

```
Command Window
|      10 |         930 |       00:15:12 |       96.88% |       81.00% |       0.0788 |       0.8409 |      1.0000e-04 |
|      10 |         950 |       00:15:29 |       96.88% |              |       0.0880 |              |      1.0000e-04 |
|      10 |         960 |       00:15:43 |      100.00% |       80.47% |       0.0018 |       0.7819 |      1.0000e-04 |
|=========================================================================================================================|
Training finished: Max epochs completed.
Test Set Accuracy = 80.47%
>>
```

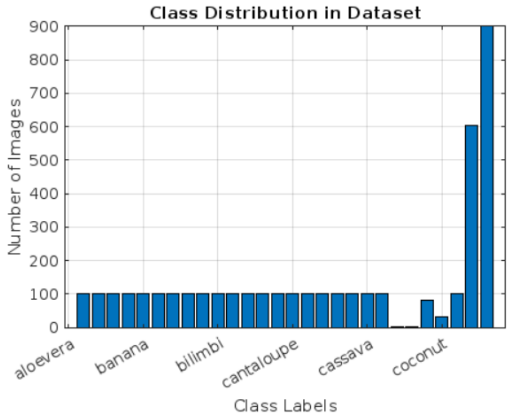
[redacted]

[redacted]

```
%% Step 8: Class Distribution Bar Plot (Test Set)
figure;
countEachLabel(imdsTest);
counts = countEachLabel(imdsTest);
bar(counts.Count)
xticklabels(counts.Label)
xlabel('Class Labels')
ylabel('Number of Images')
title('Class Distribution in Test Set')
xtickangle(45)
grid on;
saveas(gcf, 'classDistributionTest.png');
```

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]